Question Bank — Week 5

Linked lists (Lecture 05) — Answers

DSA27 · Data Structures and Algorithms

Fall 2026

Contents

Part A — Multiple choice **1**

Part B — Short answer and essay **4**

Part C — Trace the code **6**

Part D — List state **7**

Part E — Complexity analysis **7**

Part F — Find and fix the bug **8**

Part G — Write the code **8**

> ***Questions:*** `week05-questions.md`. *Commit to your own answer before reading one here — and for linked lists, draw it.*

Part A — Multiple choice

Q	Ans	Q	Ans	Q	Ans	Q	Ans
M01	a	M11	b	M21	a	M31	c
M02	c	M12	d	M22	b	M32	d
M03	b	M13	a	M23	c	M33	c
M04	d	M14	b	M24	d	M34	a
M05	a	M15	c	M25	a	M35	b
M06	c	M16	d	M26	b	M36	d
M07	b	M17	a	M27	c	M37	c
M08	d	M18	b	M28	d	M38	a
M09	a	M19	c	M29	a	M39	d
M10	c	M20	d	M30	b	M40	b

W5-M01 — a. The order is in the links themselves; there are no indices (b) and no separate order (d).

W5-M02 — c. Follow `next` i times from `head`: $O(i)$, $O(n)$ in the worst case.

W5-M03 — b. Two reference assignments, independent of n. "Amortised" (c) describes the dynamic array's append, not this.

W5-M04 — d. It must walk to the last node first. A tail reference makes it $O(1)$.

W5-M05 — a. Front insertion: two references here; every element shifts in an array.

W5-M06 — c. Indexing: address arithmetic in an array, a walk in a list. `len` (d) is $O(1)$ in both — because the list keeps `_size`.

W5-M07 — b. After `self.head = new`, `new.next = self.head` makes the node point at itself, and the old first node is no longer referenced by anything. Python evaluates each assignment's right-hand side first (a), but that is not the reason for the order of two separate statements.

W5-M08 — d. Only the predecessor holds the reference that must be redirected around the removed node.

W5-M09 — a. The head has no predecessor, so it is removed by moving `head` instead.

W5-M10 — c. Each `ll[i]` walks i steps from the head: $0 + 1 + \cdots + (n-1)$. Use `for value in ll:` for one $O(n)$ walk.

W5-M11 — b. A stored count, updated by `push_front`, `append`, `insert_at`, `pop_front` and `remove`. Forget one, and `len` lies.

W5-M12 — d. Three references — `prev`, `node` and the saved next — whatever n is.

W5-M13 — a. One 8-byte reference per slot; the node is six times larger.

W5-M14 — b. With `prev` in each node, the node itself gives you its predecessor. Indexing (a) and search (c) are still $O(n)$.

W5-M15 — c. With a permanent node before the first real one, removing the first real node looks exactly like removing any other.

W5-M16 — d. After removing the last node, `_tail` must move to its predecessor, and only a walk from the head finds it in a singly linked list.

W5-M17 — a. Blocks give array-like memory efficiency; the links give $O(1)$ at both ends.

W5-M18 — b. It is the standard off-by-one: `node.next is not None` is the right test only when you *want* to stop on the last node (to append after it).

W5-M19 — c. `c.next` is `b`, and `b.next` is `a`, whose value is 1.

W5-M20 — d. Unreachable objects are reclaimed automatically. `del` (b) only removes a *name*.

W5-M21 — a. The node at i − 1 is the one whose `next` must change. Index 0 is `push_front`.

W5-M22 — b. `ChainingHashMap` (Week 13) stores an `Array` of chains of `Entry` nodes.

W5-M23 — c. `node` is the second node, and `node.next = node.next.next` skips node 3, so the walk finds `[1, 2, 4]`. But the change bypassed the list's methods, so `_size` was never decremented and `len` still says 4. (a) is what `remove(3)` would leave. (b) skips the wrong node.

W5-M24 — d. `prev` must stop on the node **before** the gap, at index 3. It starts at index 0, so `range(index - 1)` moves it 3 times. 4 (c) is one step too far: the new node would land at index 5.

W5-M25 — a. Reading allows `0 <= index < len`, so `ll[3]` is out of range. Inserting allows `0 <= index <= len`, so (b) is a legal append at the end and (d) is `push_front`. `ll[-1]` (c) is 3: `__getitem__` adds `_size` to a negative index.

W5-M26 — b. The sentinel is permanent. It is never `None` (a) and never removed (d), and its value is meaningless (c). The first real node is `sentinel.next`, so no real node means `sentinel.next is None`.

W5-M27 — c. The node before `x` must point past it, and the node after `x` must point back past it. (a) and (b) change only `x`'s own references, which leaves the neighbours still linked to `x`. (d) makes each neighbour point at itself.

W5-M28 — d. `new.prev = a`, `new.next = b`, `a.next = new` and `b.prev = new`. A singly linked list needs only two of these, which is the "twice the bookkeeping" of the doubly linked list.

W5-M29 — a. After `pop_front`, `head` is `None` but `_tail` still refers to the old node 7. `append` sees a tail that is not `None`, so it links 5 after that dead node and never sets `head`. The walk from `head` finds nothing, while `_size` went from 0 to 1. It fails silently and raises no error (d). Every method that can empty the list must clear `_tail`.

W5-M30 — b. The body saves `following = Node(2)`, turns `Node(1).next` round to `prev`, which is `None`, then steps forward: `prev` becomes `Node(1)` and `current` becomes `Node(2)`. (d) forgets that the link was turned. (c) is the state after the **second** pass.

W5-M31 — c. `range(0 - 1)` is empty, so `prev` stays on the head, and the new node is spliced in after it, at index 1. No error is raised (b), and the list is wrong. On an empty list `prev` is `None`, and `prev.next` raises `AttributeError`. Position 0 has no node before it, so the head itself must change: that is `push_front`.

W5-M32 — d. `remove` stops at the **first** node holding the value: here it is the head (the special case), so `head` moves on and the second 4 stays. (a) would remove every 4.

W5-M33 — c. It is a constant factor that Big-O hides. An array's references are contiguous, and memory is fetched in blocks (the cache). A list's nodes were created one at a time, wherever there was room. Both walks are $\Theta(n)$ (a), and walking does not touch `_size` (b).

W5-M34 — a. Every node has a successor, so `node` is never `None` and the loop never ends. A circular walk must stop when it comes back to its starting node. Nothing in the loop shown checks for that (d).

W5-M35 — b. `a.next` is still `b`, whose value is 2. The tuple assignment sets `b.next = None` and `c.next = b`, so `c.next.value` is 2 too, and `b.next` prints `None`. (a) reads `c.next` as if it still pointed forward.

W5-M36 — d. The contract raises `IndexError("pop from empty list")`, like `list.pop()`. Without the check, `self.head.value` would raise `AttributeError` (c) on `None`, which is W5-B3.

W5-M37 — c. Inserting before a node means changing its **predecessor's** `next`, and a singly linked node has no reference back. Inserting **after** it is $O(1)$ (a). A doubly linked list makes both $O(1)$.

W5-M38 — a. Three `push_front` calls build `[2, 1, 3]` (b), since the last one pushed is first. `pop_front` takes the 2, and `append` puts it at the end: `[1, 3, 2]`. The front value has moved to the back, a rotation.

W5-M39 — d. Only the node at index 1 changes its `next`, to the new node. The new node's own `next` is set as it is made. Every node after the gap is untouched (c), which is why the splice is $O(1)$ once the walk has found `prev`.

W5-M40 — b. After reversing, the old first node is the last node. Saving `old_head = self.head` before the loop and setting `self._tail = old_head` after it costs $O(1)$, so no walk is needed (c). The last node certainly changes (a), unless the list has at most one node.

Part B — Short answer and essay

W5-E1 *(4)*

Operation	Dynamic array	Singly linked list
read element i	$O(1)$	$O(n)$
insert / remove at the front	$O(n)$	$O(1)$
append at the end	amortised $O(1)$	$O(n)$, or $O(1)$ with a tail
insert next to a node you hold	$O(n)$	$O(1)$

- **Choose the array** when you index by position, iterate a lot, or care about memory — e.g. storing measurements you read by index. It is compact and cache-friendly.
- **Choose the list** when you insert and remove at the front, or splice at positions you already hold — e.g. a stack of undo actions, the chains of a hash table. No copying, ever.

W5-E2 *(4)*

- Step 1: `new.next = self.head` — the new node points at the current first node.
- Step 2: `self.head = new` — the list now starts at the new node. Then `_size += 1`. Two assignments: $O(1)$.
- Diagram: new node → old first node → … → `None`, with `head` moving from the old first node to the new one.
- **Reversed order:** `self.head = new` first, then `new.next = self.head` sets `new.next = new` — a node pointing at itself. Nothing refers to the old first node any more, so the rest of the list is lost, and iterating loops for ever over the one node.

W5-E3 *(3)*

- Array indexing is address arithmetic, $O(1)$. List indexing must follow `next` from `head` i times, $O(i)$ — the same interface hides a different cost.

- In the loop, each `ll[i]` starts again from the head: $0 + 1 + \cdots + (n-1) = O(n^2)$ for what should be one pass.
- Write `for value in ll:` — `__iter__` keeps its place and walks once: $O(n)$.

W5-E4 *(4)*

- **Benefit:** append links after `_tail` and moves `_tail`: $O(1)$ instead of a walk.
- **Cost:** one more reference to keep correct in every method that can change the last node: `push_front` and `append` on an **empty** list (the new node is also the tail), `insert_at` at the end, `pop_front` of the **only** node, `remove` of the last node, and `reverse` (head and tail swap).
- **Removing the last node** is still $O(n)$: `_tail` must move to the node before it, and in a singly linked list only a walk from the head finds that node.

W5-E5 *(3)*

- **Doubly linked:** each node also references its predecessor. Removing a node you hold, and removing from the end, become $O(1)$; the list can be walked backwards. Cost: one more reference per node, and every insertion and removal updates twice as many links.
- **Sentinel:** a permanent node before the first real one, so every real node has a predecessor and there is no special case for the head (or for an empty list). Cost: one extra node, and every method must remember that the first real node is `sentinel.next`.

W5-E6 *(4)*

- **Choice:** a doubly linked list, holding a reference to the current node.
- **Costs:** *next* is `current = current.next`, $O(1)$. *Previous* is `current = current.prev`, $O(1)$. *Delete the current song* is `current.prev.next = current.next` and `current.next.prev = current.prev`, $O(1)$, with the head and tail handled as special cases (or a sentinel).
- **Singly linked:** *next* is $O(1)$, but *previous* and *delete* need the node before the current one, so they mean a walk from the head, $O(n)$.
- **Dynamic array**, holding an index: *next* and *previous* are $O(1)$, but deleting shifts every later song left, $O(n)$.

W5-E7 *(4)*

- **Steps:** raise `IndexError` unless `0 <= index <= len` (`index == len` means "at the end"). If `index == 0`, call `push_front`, because there is no node before the gap. Otherwise walk `prev` from the head `index - 1` steps, then `prev.next = Node(value, prev.next)` and `_size += 1`.
- **Why `index - 1`:** only the node **before** the gap holds the reference that must change. It is at index `index - 1`, which is `index - 1` steps from the head.
- **Cost:** best case $O(1)$, at index 0 or 1. Worst case $O(n)$, at the end. The splice itself is always $O(1)$.
- **`range(index)`:** the walk goes one node too far. `insert_at(1, 9)` on `[1, 2, 3]` gives `[1, 2, 9, 3]`, with no error. At `index == len`, `prev` walks off the end to `None`, and `prev.next` raises `AttributeError`.

W5-E8 *(3)*

- **Memory:** an array slot is one 8-byte reference. A `Node` with `__slots__` is about 48 bytes on 64-bit CPython, six times as much per element, plus the value object in both cases.
- **Speed:** the array's references are contiguous and the processor reads memory in blocks (the cache). Nodes are scattered, so every `next` may jump to a new place. That is the same Big-O with a bigger constant.
- **Still the right choice:** frequent insertion or removal at the front (a stack); splicing at positions you already hold (an LRU cache, an editor's buffer); never paying for a big copy at an unlucky moment; and short chains inside a hash table.

W5-E9 *(4)*

- **The invariant:** at the start of each pass, `prev` is the head of the part already reversed (`None` at first) and `current` is the head of the part still to reverse. The loop ends when `current` is `None`, and then `self.head = prev`.
- **Each pass:** `following = current.next`, then `current.next = prev`, then `prev, current = current, following`. That turns one link round and moves one step forward.
- **Why save `following` first:** once `current.next` points backwards, it is the only reference to the rest of the list. Without the saved copy, the rest is lost, and the loop steps back onto `prev` (W5-B4).
- **Space:** three references, whatever n is, so $O(1)$ extra space and $O(n)$ time. A reversed copy built by `push_front` also takes $O(n)$ time, but it makes n new nodes, which is $O(n)$ extra space, and it leaves the original list unchanged.

Part C — Trace the code

W5-T1

```
3 2 1
```

Each value is pushed on the **front**, so the last one pushed comes first — the behaviour of a stack (Week 6).

W5-T2

```
1 3 4 None
```

`a.next = a.next.next` makes node 1 skip node 2 — exactly how `remove` works.

W5-T3. 5 nodes → **2**; 6 nodes → **3**; 1 node → **0**; `None` → **0**. It moves two nodes at a time and counts the moves: it returns n // 2, half the length rounded down, in one pass — the "fast reference" of W5-C1.

W5-T4.

After step	`prev`	`node`	Links
start	None	1	1 → 2 → 3 → None
1	1	2	1 → None; 2 → 3 → None
2	2	3	2 → 1 → None; 3 → None

After step	prev	node	Links
3	3	None	3 → 2 → 1 → None

Then `self.head = prev` = 3.

W5-T5

```
b True
```

`x.next = y` closes a loop: a → b → a → b … — a **cycle**. Iterating it would never end; W5-C2 detects it.

Part D — List state

W5-S1.

```
LinkedList([1, 2, 3])   1 → 2 → 3
push_front(0)           0 → 1 → 2 → 3
insert_at(2, 9)         0 → 1 → 9 → 2 → 3
pop_front()             1 → 9 → 2 → 3          (returns 0)
remove(2)               1 → 9 → 3              (returns True)
append(5)               1 → 9 → 3 → 5
```

`len(ll)` is **4**; `ll.find(3)` is **2**.

W5-S2. head points at the new node 5, and then `5.next = self.head` makes it point at **itself**. Nodes 3 and 7 are unreachable (and collected).

```
head → 5 → 5 → 5 → ...      (5.next is node 5 itself)
       3 → 7 → None         (unreachable: lost)
```

`list(ll)` never finishes: `__iter__` yields 5, 5, 5, … for ever. (Press Ctrl+C.)

W5-S3. 7 is not at the head, so walk with `prev`:

```
start:       head, prev → 3 → 7 → 1 → 9 → None     prev.next.value is 7: match
splice:        prev.next = prev.next.next
result:        head → 3 → 1 → 9 → None                  node 7: unreachable
```

`_size` goes from 4 to 3, and `remove` returns `True`.

Part E — Complexity analysis

W5-K1 — $\Theta(n^2)$. The i-th append walks past the i nodes already there: $0+1+\cdots+(n-1) = n(n-1)/2$. **In $\Theta(n)$:** `push_front` every value ($O(1)$ each) and then `reverse()` once ($O(n)$) — or keep a tail reference.

W5-K2 — $\Theta(n^2)$. The `for` loop is one walk, n iterations; each `find(value)` walks from the head to that value's position: $1 + 2 + \cdots + n$ comparisons in total.

W5-K3. Make the **head** the front (dequeue there) and the **tail** the back (enqueue there). Enqueue: link after `_tail`, $O(1)$. Dequeue: `pop_front`, $O(1)$. The other way round, dequeue would have to remove the **last** node, which is $O(n)$ in a singly linked list (W5-M16).

Part F — Find and fix the bug

W5-B1. `while node.next is not None` stops **on** the last node without checking it: `find` of the last value returns –1. On an empty list, `node` is `None` and `node.next` raises `AttributeError`. **Fix:** `while node is not None:`.

W5-B2. It never checks the **head** itself: `remove` of the first value returns `False` and leaves it in place. **Fix:** handle the head first —

```
if self.head is not None and self.head.value == value:
    self.head = self.head.next
    self._size -= 1
    return True
```

— or use a sentinel.

W5-B3. Two bugs: on an empty list `self.head.value` raises `AttributeError` instead of the promised `IndexError`; and `_size` is never decreased, so `len` becomes wrong. **Fix:** `if self.head is None: raise IndexError(...)` first, and `self._size -= 1`.

W5-B4. `node = node.next` runs **after** `node.next = prev`, so it moves to `prev` — backwards — instead of on. On 1 → 2 → 3 → 4, the first step sets `1.next = None` and then `node = None`: the loop ends, `head` becomes node 1, and the list is just `[1]`; nodes 2, 3, 4 are lost. **Fix:** save the next node first:

```
while node is not None:
    following = node.next
    node.next = prev
    prev = node
    node = following
```

Part G — Write the code

W5-C1 — slow and fast references.

```
def middle_value(head):
    if head is None:
        raise ValueError("empty chain")
    slow = fast = head
```

```
    while fast is not None and fast.next is not None:
        slow = slow.next                    # one step
        fast = fast.next.next               # two steps
    return slow.value
```

When `fast` reaches the end, `slow` has gone half as far.

W5-C2 — Floyd's "tortoise and hare".

```
def has_cycle(head):
    slow = fast = head
    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:                    # the fast one lapped the slow one
            return True
    return False
```

Why they meet: without a cycle, `fast` reaches `None`. With one, both references end up inside the loop; each step the distance from `fast` to `slow` along the loop shrinks by exactly one, so it reaches 0 within one lap. Note `is`, not ==: the question is whether they are the **same node**.

W5-C3 — a sentinel makes the head no special case.

```
def merge_sorted_chains(first, second):
    dummy = Node(None)
    tail = dummy
    while first is not None and second is not None:
        if first.value <= second.value:   # <= keeps it stable
            tail.next, first = first, first.next
        else:
            tail.next, second = second, second.next
        tail = tail.next
    tail.next = first if first is not None else second
    return dummy.next
```

The final line links the remaining chain in one step — no loop needed, unlike an array merge. $O(n+m)$ time, $O(1)$ extra space.

W5-C4

```
def remove_duplicates_sorted(head):
    node = head
    while node is not None and node.next is not None:
        if node.next.value == node.value:
            node.next = node.next.next    # skip the duplicate; stay on node
        else:
            node = node.next
    return head
```

Staying on node after a removal matters: the new `node.next` may be another duplicate.

W5-C5 — two references k apart.

```
def kth_from_end(head, k):
    if k < 1:
        raise IndexError(k)
    lead = head
    for _ in range(k):                    # move lead k nodes ahead
        if lead is None:
            raise IndexError(k)
        lead = lead.next
    trail = head
   while lead is not None:              # move both until lead falls off the end
        lead = lead.next
        trail = trail.next
    return trail.value
```

When `lead` is `None`, `trail` is exactly k nodes before the end.